

Critical Thinking Module 3: Large Data Files Case Study

Alexander Ricciardi

Colorado State University Global

CSC507: Foundations of OSs

Professor: Dr. Joseph Issa

May 31, 2026

Critical Thinking Module 3: Large Data Files Case Study

Large data files are common in modern computing rather than being unusual edge cases. Financial, transportation, logistics, healthcare, and government databases all depend on files that can contain millions, billions, or even trillions of records. Once a file reaches that scale, processing speed becomes an operating-systems issue as much as a programming issue. The Operating System (OS) must coordinate CPU time, memory use, storage access, buffering, and concurrent work so that the file can be read and processed efficiently. This paper explores three real-world examples of large data files having billions of rows and explains why it is important for businesses and institutions to process those files in fractions of a second instead of several seconds. The three examples are financial market audit-trail files, package-tracking and inventory-movement files, and driver's-license or vehicle-record files. Additionally, this paper explores how the examples can be connected to OS concepts and data processing, such as streaming, chunking, buffering, timing, and multithreading or parallelism.

Financial Market Audit-Trail Files

One financial example of a massive data file is financial market audit-trail data. The U.S. Securities and Exchange Commission (SEC, 2017) through Rule 613 created the Consolidated Audit Trail (CAT) that allows regulators to track activity across the U.S. equity and options markets. The file captures events throughout the order life cycle, including origination, routing, modification, cancellation, and execution. The same rule also requires detailed timestamps, which show that this data is not simply archived for long-term reference, but it is also operational data that supports real oversight and event reconstruction. The scale of this file is extremely large. An SEC-hosted filing reported that the CAT was originally expected to handle more than 58 billion records per day, but first-quarter 2025 activity averaged 752 billion events per day,

with a peak of 1.45 trillion events on April 7, 2025 (SEC, 2025). This means the one-million-row threshold is passed almost immediately. It also means that any repeated full-file scan becomes expensive very quickly.

It is important for a business or regulatory system to process this kind of data in fractions of a second because financial conditions change very quickly. A slow query, correlation step, or anomaly check can delay fraud detection, market surveillance, risk review, and trade reconstruction. If one request takes several seconds, that delay multiplies when many analysts, automated systems, or regulators issue related queries at the same time. By the time a slow process finishes, the market may already have moved into a different state. For this reason, the practical value of the data depends heavily on low-latency processing. OS-level processing principles apply directly to this kind of file. A strong design would stream the records sequentially, divide the work into chunks, and let separate workers process independent sections. This is especially important on modern multicore processors because the scheduler can keep several workers active at once instead of forcing one thread to handle the entire workload alone. The benefit is not only raw parallelism. When one worker is delayed by storage access, another worker may still be able to use the CPU. At the same time, the system cannot simply create unlimited threads because too many workers can increase contention and reduce the benefit. Therefore, chunk size, thread count, and storage throughput all matter. If repeated lookups are common, sorted structures and indexes can reduce the need to rescan the raw file over and over again.

Package-Tracking and Inventory-Movement Files

A second real-world example is a package-tracking or inventory-movement file. UPS reports that it delivered an average of 20.8 million packages per day in 2025, for a total of 5.2

billion packages during the year (UPS, n.d.). Even if a company stored only one row per package, a single day's data would already be far beyond one million rows. In practice, the file would usually be much larger because each package can generate multiple events, such as label creation, intake, hub arrival, hub departure, customs clearance, vehicle loading, delivery attempt, and final delivery. This kind of file must be processed quickly because logistics work depends on accurate timing and current system state. Sorting centers, warehouse dashboards, route planning systems, customer-support systems, and customer-facing tracking pages all depend on fast reads and updates. If a route-exception lookup or package-status request takes several seconds instead of a fraction of a second, the delay can spread into other parts of the operation. A package may be directed to the wrong path, a warehouse dashboard may display stale counts, or a customer may receive delayed information. These delays may seem small at first, but when they are repeated across millions of events, they become a serious operational bottleneck.

The optimization methods for this example follow standard large-file processing principles. The first method is to stream the file instead of loading it completely into memory. The second method is to break the file into independent parts, such as time ranges, facilities, or fixed-size blocks, so those parts can be processed in parallel. The third method is to buffer output rather than write every result line immediately. That reduces write overhead and helps the storage subsystem stay efficient. This example also benefits from a better interpretation of what "parallel" really means. If the workload is mostly CPU-based, then multiple threads can improve throughput by keeping several cores busy. If the workload is mostly waiting on disk access, the gain may come from overlapping I/O waits with useful work on other chunks. However, if the file is split too aggressively, the storage device can become the bottleneck and the improvement may flatten out. The logical conclusion is that concurrency helps most when it is matched

carefully to both the processor and the storage system, not when threads are added without measurement.

Driver's-License and Vehicle-Record Files

A third example is a driver's-license or vehicle-record file maintained by a state agency. The California DMV reports 35,149,648 DL/ID cards and 36,190,654 registered vehicles as of January 1, 2026 (California DMV, 2026a). The DMV also states that it maintains public records involving vehicle registration, driver's licenses, occupational licenses, and related statistical information (California DMV, 2026b). This shows that a statewide file can already contain tens of millions of rows before renewals, suspensions, convictions, insurance checks, address changes, and other related events are added. This kind of data must be processed quickly because it supports both public service and administrative decision-making. These files are used for license issuance, renewals, record updates, eligibility checks, identity verification, employer checks, enforcement actions, and law-enforcement access. If a record lookup or update takes several seconds during normal operation, service lines become longer, online systems feel slower, and safety-related checks may be delayed. The effect is not limited to user convenience. It also affects throughput across the entire service environment.

The optimization strategy for this example depends on the task. If the job is a full-file audit, then sequential streaming is a strong choice because it reduces unnecessary random disk access. If the job is repeated point lookup by driver's-license number or vehicle ID, then a sorted structure or index is much more efficient than repeated linear scans. This distinction matters because different workloads stress different OS resources. A full audit places more pressure on sustained I/O throughput, while repeated lookups place more pressure on search efficiency and latency. A second useful improvement is to separate live transaction work from reporting work.

The active system should handle quick targeted lookups and updates, while heavier analytical reporting should run on snapshots, replicas, or partitions designed for bulk reads. That reduces lock contention and protects the responsiveness of the live service path. This is similar to the distinction between interactive and background work in process management. The operating system and the application both perform better when the most urgent tasks are not forced to wait behind large reporting jobs.

Why Fractions of a Second Matter

Across all three examples, the difference between fractions of a second and several seconds matters because the same work is repeated many times. One slow request may not seem serious by itself, but a delay that small becomes expensive when it happens thousands or millions of times per day. In finance, it delays surveillance and risk review. In logistics, it slows tracking, routing, and customer communication. In public record systems, it reduces service quality and delays administrative decisions. Slow processing also uses system resources inefficiently. When each request runs longer than necessary, CPUs remain occupied longer, I/O queues stay active longer, and more tasks build up in the system. Guo et al. (2024) explain that process scheduling aims to optimize processor utilization, response time, and throughput, while synchronization and resource management help multiple processes interact correctly. That idea applies directly here. Faster file processing is not only a convenience for the user. It is an example of better resource coordination by the overall system.

OS Concepts for Large Data Files

OS concepts, such as multithreading, buffering, and stream processing, provide an approach that could be used to improve the processing of large data files. First, the system should stream the file rather than attempt to load the file into RAM. This keeps memory use bounded

and lowers the risk of swapping or memory pressure. Second, the work should be divided into chunks rather than the entire file as a single block. That is, if a file can be partitioned safely, then several threads or workers can process those sections at the same time. This is very useful for systems that can utilize multiple cores or multithreading because the scheduler can distribute runnable work across more execution contexts. The gain does not come from the CPU “doing more than one thing magically.” It comes from the OS having more opportunities to keep work moving while other tasks are waiting. Third, disk access should be kept organized as much as possible. Sequential reads are usually more efficient than repeated random reads. When repeated lookups are necessary, using an index or a sorted file can lower latency and reduce redundant scanning.

Fourth, output should be buffered, as writing every line immediately, that is, without buffering, increases overhead because it increases how frequently the writing process is activated. Additionally, buffered output collects results into larger groups, improving throughput and lowering pressure (writing frequency) on the storage subsystem. Fifth, each implementation version should be timed. Measuring the run time of a single-thread, a chunked or multithreaded version, and a split-file parallel version makes it possible to compare throughput, scaling behavior, and latency. This provides evidence instead of an assumption of performance or speedup. Finally, correctness must be verified after optimization. Faster output is not useful if the answer is wrong. Line counts, sample checks, totals, and merge validation are all important after the file has been divided or processed in parallel.

Conclusion

The case studied in this paper show that, in many real-world systems, million-row data files are normal. For example, in financial market audit-trail files, package-tracking files, and

driver's license or vehicle-record files that exceed one million rows, highlighting why several seconds of delay can become a major problem. In finance, slow processing reduces the value of oversight data. In logistics, it interferes with tracking, routing, and customer service. In government record systems, it reduces responsiveness and slows high-volume administrative work. The main conclusion is that large-file processing should be treated as an operating-systems problem as well as a programming problem. Streaming, chunking, buffering, indexing, concurrency, and timing are practical tools for improving how the system uses CPU time, memory, and storage access. Efficient large-file processing depends on structured work, measurement, and correct use of system resources rather than on brute force alone.

References

- California Department of Motor Vehicles. (2026a). *DMV statistics*. State of California DMV
<https://www.dmv.ca.gov/portal/news-and-media/dmv-statistics/>
- California Department of Motor Vehicles. (2026b). *Vehicle or driver's records requests*. State of California DMV. <https://www.dmv.ca.gov/portal/customer-service/request-vehicle-or-driver-records/>
- Guo, J., Wu, S., Chen, H., Huang, S., & Zhang, Y. (2024). Application analysis of process state control in modern OS. *Academic Journal of Computing & Information Science*, 7(6), 13-19. <https://doi.org/10.25236/AJCIS.2024.070603>
- U.S. Securities and Exchange Commission. (2017, October 11). *Rule 613 (Consolidated Audit Trail)*. US SEC. <https://www.sec.gov/about/divisions-offices/division-trading-markets/rule-613-consolidated-audit-trail>
- U.S. Securities and Exchange Commission. (2025). *Securities Exchange Act Release No. 34-103374; File No. SR-MEMX-2025-20*. US SEC.
<https://www.sec.gov/files/rules/sro/memx/2025/34-103374.pdf>
- United Parcel Service, Inc. (n.d.). *Company profile*. UPS. <https://investors.ups.com/company-profile>